2) Текст, поетапна розповідь по коду для захисту

Ось готовий текст, який можна читати майже як відповідь на захисті.

Загальне пояснення

У цій лабораторній роботі потрібно було використати генератори списків для обробки формалізованих даних. За умовою мого варіанту, на ділянку можна посадити дерева трьох найменувань, якщо всі вони пристосовані до одного й того ж клімату і належать до одного типу, тобто є або листовими, або хвойними, або плодовими. Для цього я створив програму мовою Erlang, у якій дерева описані у вигляді списку кортежів, а пошук відповідних груп виконується за допомогою генераторів списків.

Пояснення модуля

На початку програми я оголошую модуль:

```
-module(lab7_var16).
```

Це означає назву мого модуля. Далі я вказую експортовані функції:

```
-export([can_plant/0, db_trees/0]).
```

Це означає, що з консолі можна викликати дві основні функції. Функція `db_trees/0` повертає базу даних дерев, а функція `can_plant/0` виконує основну логіку задачі.

Пояснення бази даних

Далі йде функція `db_trees/0`. У ній я задаю базу даних у вигляді списку кортежів формату `{Name, Climate, Type}`. Перший елемент кортежу — це назва дерева, другий — клімат, до якого воно пристосоване, а третій — тип дерева. Наприклад, запис `{oak, temperate, deciduous}` означає, що дуб є листовим деревом і пристосований до помірного клімату.

У базі є кілька груп дерев. Наприклад, листові дерева для помірного клімату, хвойні дерева для холодного клімату, плодові дерева для помірного клімату та хвойні дерева для сухого клімату. Саме ці записи потім використовуються для формування відповідей.

Пояснення функції can_plant/0

Основна функція програми — це `can_plant/0`. Спочатку вона зчитує базу даних:

```
DB = db_trees(),
```

Тобто змінна `DB` отримує весь список дерев.

Після цього створюється список `Groups0`. Тут використовується генератор списків. Його завдання — для кожної комбінації клімату і типу знайти всі дерева, які мають саме ці характеристики. У результаті формується кортеж виду `{Climate, Type, [Names]}`. Тобто ми отримуємо клімат, тип дерева і список назв дерев, які підходять під ці дві ознаки.

Наприклад, для `temperate` і `deciduous` формується список `[oak, maple, birch, linden]`. Для `cold` і `coniferous` формується список `[pine, spruce, fir]`.

Пояснення генератора списків

У середині `Groups0` є вкладений генератор списків:

```
[Name || {Name, C, T} <- DB, C == Climate, T == Type]
```

Його робота така: він проходить по всіх елементах списку `DB` і вибирає тільки ті дерева, у яких клімат збігається зі змінною `Climate`, а тип збігається зі змінною `Type`. З цих записів у новий список додається тільки назва дерева `Name`. Саме так формується список імен для кожної групи.

Пояснення видалення дублікатів

Оскільки генератор списків проходить по кожному елементу бази, одна й та сама група може з’явитися кілька разів. Тому після побудови `Groups0` я викликаю функцію:

```
Groups = del_dbles(Groups0),
```

Функція `del_dbles/1` видаляє дублікати груп. Вона бере першу групу зі списку, запам’ятовує її клімат і тип, а потім за допомогою функції `del_examples/2` прибирає всі інші елементи з такими самими значеннями.

Пояснення функції del_dbles/1

Функція `del_dbles/1` працює рекурсивно. Якщо список не порожній, вона бере його голову, тобто перший елемент, і хвіст, тобто решту списку. Потім із хвоста видаляються всі дублікати поточної групи. Після цього поточна група додається у результат, і функція рекурсивно обробляє очищений хвіст. Якщо список порожній, повертається порожній список.

Пояснення функції del_examples/2

Функція `del_examples/2` теж працює рекурсивно. Вона проходить по списку груп і порівнює пару `{Climate, Type}` з ключем, який їй передали. Якщо пара збігається, цей елемент пропускається. Якщо не збігається, елемент зберігається у списку результату. Таким чином прибираються всі дублікати поточної групи.

Пояснення фінального відбору

Після видалення дублікатів у функції `can_plant/0` виконується ще один генератор списків:

```
[G || G = {_Climate, _Type, Names} <- Groups, length(Names) >= 3].
```

Його завдання — залишити тільки ті групи, у яких кількість назв дерев не менша за три. Це відповідає умові задачі, де сказано, що на ділянку можна посадити дерева трьох найменувань. Якщо в групі менше трьох дерев, вона не потрапляє у фінальний результат.

Що означає результат роботи програми

Результат програми — це список кортежів. Наприклад:

```
{temperate,deciduous,[oak,maple,birch,linden]}
```

Це означає, що для помірного клімату є група листових дерев, які можна висаджувати на одній ділянці. Аналогічно, запис

```
{cold,coniferous,[pine,spruce,fir]}
```

означає, що для холодного клімату знайдено групу хвойних дерев.

Загальний висновок для захисту

Отже, у цій програмі я представив дані у вигляді списку кортежів, використав генератори списків для формування груп дерев за кліматом і типом, реалізував видалення дублікатів і відбір лише тих груп, які задовольняють умову задачі. У результаті програма знаходить усі варіанти дерев, які можна посадити на одній ділянці відповідно до мого варіанту.